

Consolidated Architecture for Phase 1

A source-of-truth-aligned architecture brief for the OpenAI Build Week MVP. This is a design deliverable only; no product implementation is authorized by this document.

DECISION STATUS

Proposed. The Project Bible is the governing source. The technical design, master engineering prompt, and revision specification contribute implementation detail only when they do not conflict with the Bible.

The product promise

In one session, an international graduate can upload a resume, receive an explained career assessment, discover sponsorship-aware roles, produce editable application material, practise an interview answer, and leave with a four-week action plan.

Project Bible v1.0	Source of truth for mission, MVP scope, canonical routes, FastAPI boundary, data model, API envelope, and delivery constraints.
Technical Design Document	Adds ESP-centric orchestration, strict structured-output contracts, auditability, failure handling, seed-data strategy, and UI trace detail.
Codex Master Engineering Prompt	Adds stack preferences, folder concepts, milestones, and development discipline.
Engineering Revision Specification v2	Editorial guidance only; it must not supersede product or architecture decisions.

Prepared 17 July 2026 | Phase 1 approval gate

1. Product vision and MVP boundary

CareerPilot OS is an AI-native career operating system for international students seeking sponsored graduate roles. Its job is not to automate applications; it is to turn candidate evidence into an explainable priority and a practical next action.

Resume upload and analysis; curated job discovery and ranking; tailored resume content and cover letters; interview practice and feedback; application tracking; four-week strategy.	Automatic job applications; fragile live scraping; legal immigration advice; video interview analysis; native mobile apps; teams; payments.

Phase 1 north-star behavior

First 3 minutes	Credible Career Health assessment	PDF extraction, validation, persisted resume evidence, visible analysis state.
Every job-fit score	Reasons, gaps, sponsorship confidence, next action	Typed recommendation records; no raw model prose as a decision.
Application tailoring	Editable variant without overwriting original	Append-only resume versions; original is immutable.
Three-minute demo	Reliable core journey with seeded data	Curated jobs and demo account; no live scraping dependency.

Canonical routes

/login /register /dashboard /resume /jobs /applications /interview /strategy /settings

PRODUCT INVARIANT

No score is shown without contributing factors and a clear action. Career Health remains a core user-facing assessment; the richer ESP concept from the technical design is treated as a proposed aggregate only after its relationship to the Bible is approved.

2. Governing decisions and source conflicts

The documents agree strongly on the product and AI safety model. The meaningful divergences are architectural and should be made explicit before implementation.

Backend boundary	Next.js frontend calls a separate FastAPI API; all routes are /api/v1.	Single Next.js app with Route Handlers; no separate backend.	Use the Bible: separate FastAPI service. Frontend never owns prompts, secrets, or orchestration.
Validation / data access	Pydantic and Supabase; no ORM prescribed.	Zod + Drizzle as source of truth.	Use Pydantic for API/agent contracts. Decide SQLAlchemy versus direct Supabase access in Phase 1; do not introduce Drizzle into a Python API without an explicit change request.
North-star metric	Career Health is the named assessment and API result.	ESP is the system-wide 0-100 outcome metric; Career Health is one component.	Keep Career Health canonical. Add ESP only as a documented extension after the owner approves formula, weights, and UX naming.
Agents	Coordinator plus Resume, Job, Strategy, Interview, Learning, Application.	Coordinator plus Resume, ATS, Job, Strategy, Interview, Learning.	Retain the Bible six. Treat ATS analysis as a Resume/Application capability, not a seventh top-level agent.
API shape	{data, meta}; errors {error:{code,message,details}}.	Unversioned /api endpoints; a different error envelope; SSE for compute.	Retain Bible versioning and envelope. Add SSE only as an optional response mode on /api/v1/career/analyze or a dedicated orchestration endpoint.
Deployment	Not fixed.	One Vercel project / serverless Route Handlers.	Vercel is suitable for frontend. FastAPI hosting is a Phase 1 deployment decision, not silently replaced by Next.js Route Handlers.

RESOLUTION RULE

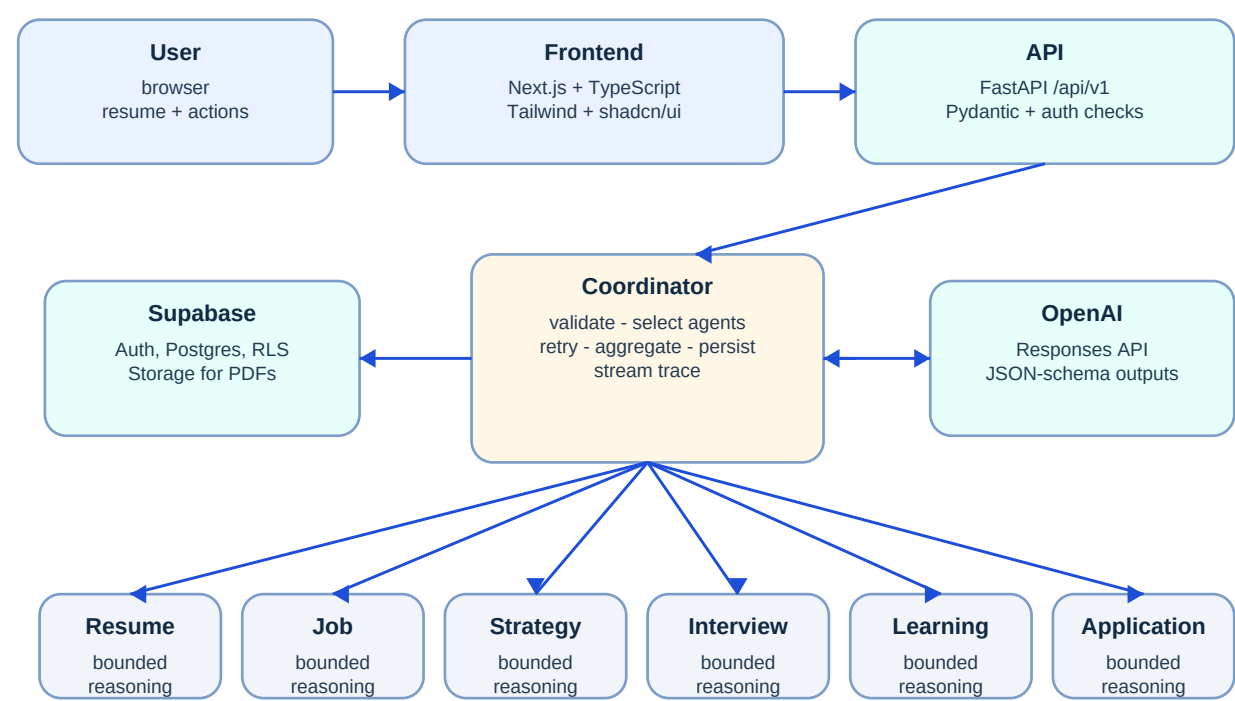
Where a later document calls itself authoritative, it is subordinate to the explicit instruction that the Project Bible is the source of truth. This brief preserves useful later-document controls only when compatible with that rule.

3. Whole-system architecture

The recommended baseline preserves the Bible's client/API separation and applies the technical design's strongest control: deterministic orchestration with validated, evidence-bound model outputs.

Source-of-truth Phase 1 architecture

Browser UI and FastAPI are separate trust boundaries; the coordinator remains deterministic application code.



Evidence	Resume versions, profile preferences, curated job facts, application records.	Sponsorship status is sourced from curated data and is never inferred.
Control	FastAPI services and deterministic Coordinator sequence work, validate outputs, persist records, and emit progress.	No LLM decides workflow transitions, weights, retries, or writes.
Reasoning	Six specialist agents interpret only the bounded input supplied to them.	Every recommendation is grounded in supplied evidence and validates against a schema.
Outcome	Career Health, job fit, next actions, variants, strategy, and interview feedback in the UI.	No unexplained score or recommendation is rendered.

4. Component ownership and trust boundaries

Next.js frontend	Rendering, interaction state, client validation, authenticated API calls, safe optimistic UI, loading/empty/error states.	Prompt text, service-role credentials, authorization decisions, PDF processing, database writes outside approved API routes.
FastAPI API	Bearer-token validation, authorization, PDF extraction, versioned REST contract, orchestration invocation, persistence, logging, observability.	Trusting user IDs from the client or returning raw model text as product decisions.
Coordinator	Minimum-context loading, dependency ordering, specialist selection, retry policy, schema/domain validation, aggregation, audit history, trace events.	Free-form career advice, legal conclusions, model-led routing, or uncontrolled tool access.
Specialist agents	Bounded resume/job/strategy/interview/learning/application reasoning under a typed task.	Persistence, mutable state, score weights, external job discovery, or sponsorship claims beyond source data.
Supabase	Auth, Postgres, Storage, Row Level Security, source facts and durable domain state.	Interpretation of policy, resume facts, or model recommendations.

Request path

Authenticate -> validate input -> store/locate immutable evidence -> Coordinator builds minimal context -> invoke specialists -> validate structured results -> persist history and domain output -> return a versioned API result (and, if approved, stream trace progress).

WHY THE COORDINATOR IS CODE

It must apply identical state transitions, bounded retries, deterministic arithmetic, and transactional writes for identical validated inputs. These are control-plane obligations, not language-model decisions.

5. AI system and prompt contract

OpenAI Responses API calls are isolated behind the backend. Each specialist receives the least context needed and returns JSON that is validated before it can reach the UI or affect a stored score.

Resume	Resume text, target role	Parsed profile, Career Health dimensions, evidence-backed resume and portfolio recommendations.
Job	Candidate profile, filters, curated jobs	Ranked matches, fit reasons, missing skills, factual sponsorship compatibility, next action.
Strategy	Health, matches, goals	Feasible four-week roadmap tied to identified gaps.
Interview	Role, job, answer	Relevant question, STAR evaluation, feedback, retry action.
Learning	Verified skill gaps, available time	Learning resources, project ideas, sequence; advisory support.
Application	Job, dates, current status	Pipeline advice, deadline and follow-up actions.

Shared recommendation contract

evidence	Specific supplied resume, job, or curated-data fact; never invented.
reasoning	Connects evidence to the recommendation; it does not masquerade as fact.
impact	Names the affected assessment dimension or explicitly marks the item advisory.
next_action	Exactly one concrete, feasible action.
confidence	Numeric confidence with a low-confidence visual treatment where relevant.

PROMPT STANDARD

Every prompt includes role, permitted inputs, task, rubric, safety constraints, output JSON schema, source-grounding rules, and a prohibition on invented evidence. API/UI compose display prose from validated fields.

6. Data architecture and lifecycle

All primary records use UUIDs, created_at, and updated_at. User-owned tables enforce Row Level Security by user_id. Resume versions are append-only: an edit produces a new record, never an overwrite of the original.

Identity and profile	profiles	Target role, country, sponsorship need, and skills belong to the signed-in user.
Evidence	resumes	Original file path, source text, parsed JSON, version name, is_original. Secure storage and immutable originals.
Assessment	career_scores	Overall + dimensions + analysis JSON, linked to the resume used.
Discovery	jobs	Curated title, company, location, skills, sponsorship_status, source URL; demo data is frozen.
Progress	applications, interviews, learning_paths, activity_logs	Capture user actions, status changes, plans, and feedback.
Audit	agent_history	Agent name, normalized inputs, validated outputs, execution status; access-controlled.

Proposed ESP extension (approval required)

The technical design introduces a reproducible aggregate with seven weighted components: Resume Quality, Career Health, Skills Match, Portfolio Strength, Interview Readiness, Application Quality, and Sponsorship Compatibility. If adopted, store raw component values, contribution, immutable weight-version, and the originating agent record. Never hide a partial failure as a complete score.

RLS BASELINE

The database must fail closed: every user-owned query, storage object, and audit record is scoped to the authenticated subject. The API validates identity independently and never trusts a client-supplied user_id.

7. API, events, and interface contracts

The Bible's API convention is retained. REST routes live under /api/v1; success responses use { **data**, **meta** }, errors use { **error**: { **code**, **message**, **details** } }, and every user route requires a validated bearer token.

Health	GET /api/v1/health	Service status; no user data.
Resume	POST /api/v1/resume/upload; POST /api/v1/resume/analyze	Multipart PDF -> secure original and extraction status; CareerHealth result is explainable.
Jobs	POST /api/v1/jobs/search	Role, country, skills, sponsorship -> ranked JobMatch[] with factual sponsorship confidence.
Tailoring	POST /api/v1/resume/tailor; POST /api/v1/cover-letter	Creates a preserved variant and editable, evidence-based content.
Interview	POST /api/v1/interview/start; POST /api/v1/interview/evaluate	Returns a question/session then structured STAR feedback.
Strategy	POST /api/v1/career/strategy	Profile + resume -> four-week roadmap.
Dashboard / pipeline	GET /api/v1/dashboard; GET/POST /api/v1/applications	Aggregated daily command centre and application state.

Optional streaming adapter

If live orchestration is approved for the demo, use Server-Sent Events from a dedicated FastAPI endpoint or a negotiated stream mode. Event types are: agent_start, agent_result, agent_retry, agent_failed, assessment_computed, done. Streaming changes transport only; it does not alter the versioned response schema or permit raw output through.

8. Frontend architecture and UX

App shell	Consistent responsive navigation and page outlet.	AppShell, Sidebar, Topbar; persistent desktop sidebar and mobile drawer.
Shared primitives	Accessible reusable visual states and score explanation.	MetricCard, ScoreRing, EmptyState, LoadingSkeleton, ErrorState, ReasonList, StatusBadge, ActionCard.
Feature domains	Route-local UI and data behavior.	resume, jobs, applications, interview, strategy, dashboard, settings.
Data access	Authenticated API calls, cache, safe mutations, client validation.	Typed client generated/maintained from OpenAPI; React Query is compatible but remains a frontend detail.

Design rules

- Modern, calm SaaS UI; dark mode and responsive layout from day one.
- One primary action per screen, with explicit loading, empty, error, and permission states for every async surface.
- Career Health status labels pair semantic language with colour: Strong, Developing, At risk.
- Scores must link to reasons, gaps, and a next action; never rely on colour alone.
- Accessibility baseline: keyboard navigation, semantic controls, visible focus, labelled charts, and adequate contrast.

VISIBLE ORCHESTRATION

The technical design's live trace is a valuable demo and trust feature. It should show actual backend stage events, not simulated progress; static loading states remain correct when streaming is unavailable.

9. Consolidated repository structure

This structure expands the Bible's folders without changing its split frontend/backend architecture. It is a planning target, not an instruction to create files yet.

```
careerpilot-os/  
  frontend/  
    app/ # routes and layouts  
    components/ # ui, shared, page primitives  
    features/ # resume, jobs, applications, interview, strategy, dashboard  
    hooks/ services/ types/ styles/  
    public/  
  backend/  
    app/  
    api/v1/ # FastAPI routers  
    agents/ # six bounded specialists  
    coordinator/ # deterministic workflow and event trace  
    services/ repositories/ schemas/ prompts/  
    database/ core/ utils/  
    main.py  
    tests/  
  supabase/ # migrations, RLS policies, seed data  
  docs/  
  README.md
```

Contract sharing

FastAPI OpenAPI is the cross-boundary source. The frontend uses generated or explicitly maintained TypeScript types from that contract. Pydantic models and OpenAPI protect server correctness; client-side schema validation may mirror only user-facing forms to preserve a coherent UX.

SECRET HYGIENE

Only backend environment variables may contain OpenAI credentials or Supabase service-role credentials. Commit `.env.example` only. Prompt text is backend-owned and never shipped to the browser.

10. Engineering constraints and quality gates

Typing and contracts	TypeScript strict mode; Python type hints; stable OpenAPI-derived/shared API types; external input validated.	Typecheck and contract tests.
Privacy	Secure original resumes, scoped storage, RLS, safe logs; no resume text or token content in ordinary application logs.	RLS and authorization integration tests; log review.
Explainability	Evidence -> Reasoning -> Impact -> Next Action for every shown recommendation; suppress incomplete results.	Schema tests and UI render gate.
Sponsorship safety	Do not fabricate eligibility or availability; surface only curated/explicit source status with confidence language.	Fixture and regression tests against seed facts.
Failure handling	Bounded retry; explicit failed/degraded states; no fabricated completion or silent zero score.	Failure-mode integration tests.
Testing	Unit tests for transformations/repositories; auth and core-route integration tests; one complete seeded-account E2E demo.	Pre-demo test checklist.
Demo reliability	Seed jobs and a demo account; no fragile live job integration; keep the core journey demonstrable in 3 minutes.	Rehearsal against frozen demo data.

Observability baseline

Attach a request ID to inbound API calls and downstream agent runs. Log route, latency, status, safe failure class, agent name, model/prompt/schema version, and token/cost metadata where available. Store user-visible agent history under RLS; never expose secrets or raw resume text in general logs.

11. Recommended Phase 1 sequence

The implementation order is intentionally vertical. It creates a credible demo early, then adds breadth only after contracts, persistence, and failure behavior are proven.

0. Architecture decision	Confirm the Bible's FastAPI split, whether ESP is added now, and deployment target for FastAPI.	Written approval of this brief and listed decisions.
1. Foundation	Repository shell, Supabase project/schema/RLS, auth boundary, FastAPI health route, frontend navigation/theme.	Health, auth boundary, and database policy checks work.
2. Resume-to-Health slice	Upload a PDF, extract safely, persist original/version, run Resume Agent, return explainable Career Health.	First-time user sees a credible result under three minutes.
3. Job-to-application slice	Curated jobs, explainable ranking, immutable resume variant, editable cover letter, application entry.	One job lifecycle works with sponsorship facts.
4. Strategy and interview	Four-week plan, interview question/evaluation, activity updates.	Recommendations remain evidence-backed and actionable.
5. Reliability and demo	Tests, observability, seed account, UX states, rehearsal, deployment, recording.	Full three-minute demo passes repeatedly.

FEATURE FREEZE

Do not add scope unless it clearly improves AI reasoning, technical difficulty, product polish, UX, real-world impact, or the demo. The core path wins over additional disconnected features.

12. Decisions needed before code

Approval of the architecture permits Phase 1 only after the following decisions are confirmed. These are the small number of choices that materially affect implementation direction.

Service topology	Next.js frontend + separate FastAPI backend, as specified by the Project Bible.	It changes the repository, deployments, auth flow, API types, and observability boundary.
Assessment naming	Ship Career Health as the MVP assessment. Defer ESP aggregation unless explicitly approved.	The documents disagree on the top-level metric and score semantics.
ATS scope	Implement as a capability inside Resume/Application workflows rather than a top-level agent.	The Bible names Application Agent; later material substitutes ATS Agent.
Persistence access	Pydantic contracts plus Supabase-backed repositories; choose ORM/query approach during Phase 1 design.	Drizzle is TypeScript-centric and conflicts with the Bible's FastAPI boundary.
Streaming	Add only if it uses actual FastAPI stage events and does not compromise the REST contract.	It is valuable for demo visibility but not required for correctness.
FastAPI hosting	Select a managed Python host appropriate to the team and deployment constraints.	Vercel alone cannot be assumed once the Bible's separate FastAPI service is retained.

Approval statement

Approve this architecture to begin Phase 1 with the stated source-of-truth resolutions, or provide revisions to the six decisions above. No implementation has been started.